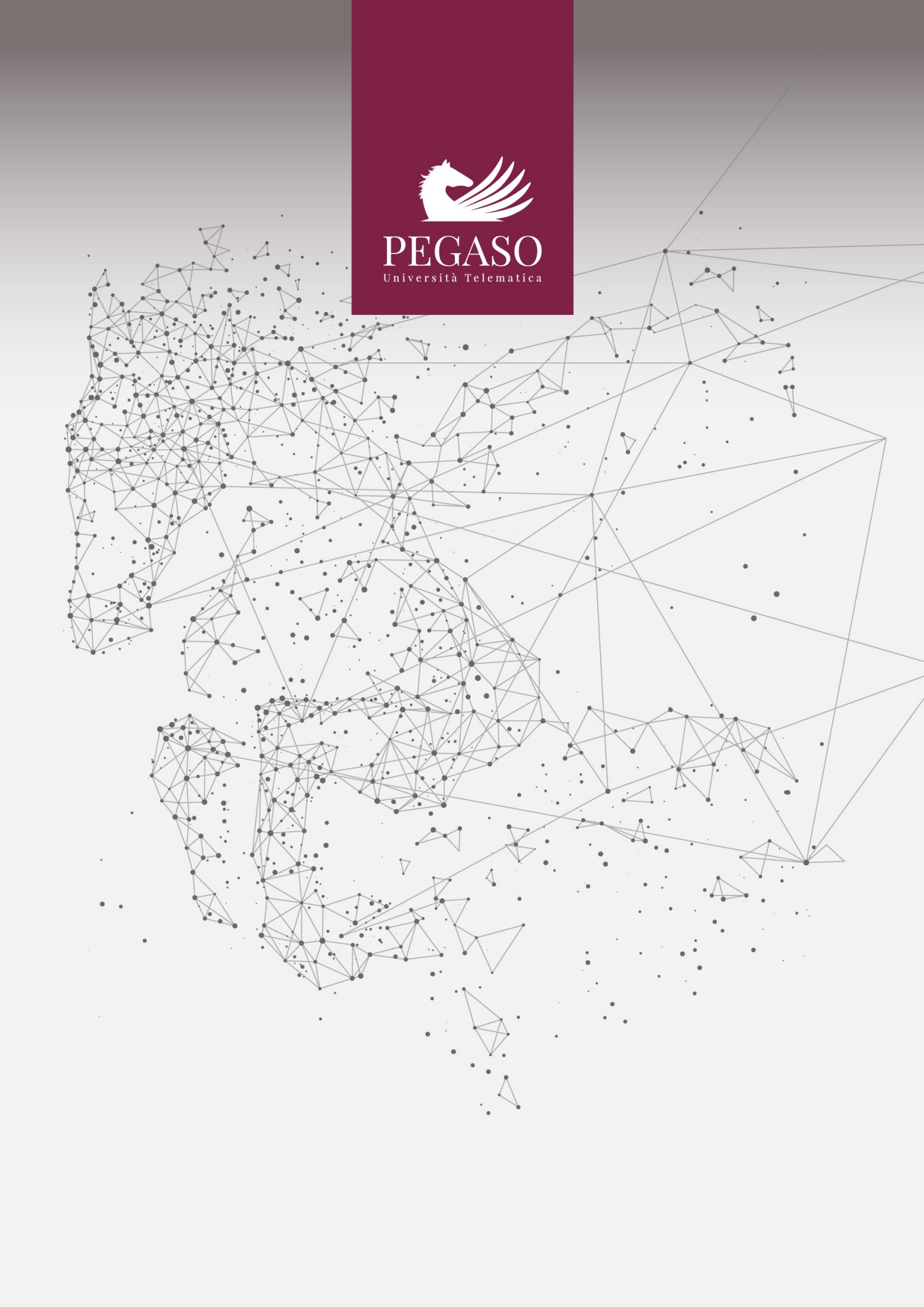

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. AMBITI DI SVILUPPO IN JAVA 3
2. CLASSE ED OGGETTI 4
3. CLASSI E OGGETTI IN UN PROGETTO IN JAVA 7
4. METODI 9
5. VARIABILI 12
BIBLIOGRAFIA 15

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Ambiti di sviluppo in Java

Prima di approcciare il codice e analizzare i componenti fondamentali di Java, approcciamo il linguaggio in modo schematico introducendo una importante distinzione. Si possono distinguere due ambiti in si agisce quando si sviluppa un'applicazione Java:

- ambito del compilatore (o compilativo, o delle classi)
- ambito della Java Virtual Machine (o esecutivo, o degli oggetti)

All'interno dell'ambito del compilatore si codifica la parte strutturale dell'applicazione. L'ambito esecutivo servirà per definire il flusso di lavoro che dovrà essere eseguito.

Distinguere questi due ambiti sarà utile per comprendere i ruoli che dovranno avere all'interno delle applicazioni i concetti e i componenti introdotti in questo modulo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Classe ed oggetti

A livello concettuale, la classe rappresenta un'astrazione di un concetto nel mondo reale (concreto o astratto).

L'approccio orientato agli oggetti è nato proprio per soddisfare l'esigenza umana di interagire con il software in maniera più naturale, con strutture dati con un contesto e significato semantico vicino alla realtà che ci circonda. Scrivere un programma in ambito bancario che riguarda conti correnti, clienti, saldi, prelievi usando solo variabili intere, stringhe o vettori può diventare assai complicato. Nell'approccio orientato agli oggetti i programmi manipolano strutture dati più complesse che rappresentano più fedelmente le entità del mondo reale (conti bancari, dipendenti, automobili, rettangoli, ecc...), denominate classe.

A livello di sviluppo software, la classe è la specifica di un'entità software che rappresenta un'astrazione indicante una categoria di oggetti che condividono le stesse caratteristiche e le stesse funzionalità. Le caratteristiche sono date dall'insieme di variabili o attributi (i cui valori correnti definiscono lo stato) mentre le funzionalità sono fruibili attraverso una serie di operazioni (metodi) che è possibile invocare sugli oggetti istanze della classe.

In fase di analisi si tratta principalmente la classe come rappresentazione di concetti laddove vi è maggiormente la necessità di chiarire gli ambiti semantici e di creare un dizionario comune fra committenti e progettisti del software.

In fase di progettazione si tratta principalmente le classi come entità software e definite con un livello di dettaglio più alto recante anche le specifiche sugli attributi e i metodi.

L'oggetto è l'istanza (ovvero la realizzazione fisica) di una classe.

In Fig. 1 è mostrata la sintassi per la dichiarazione di una classe tramite un esempio relativo ad una classe che rappresenta un'astrazione del concetto di Auto.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

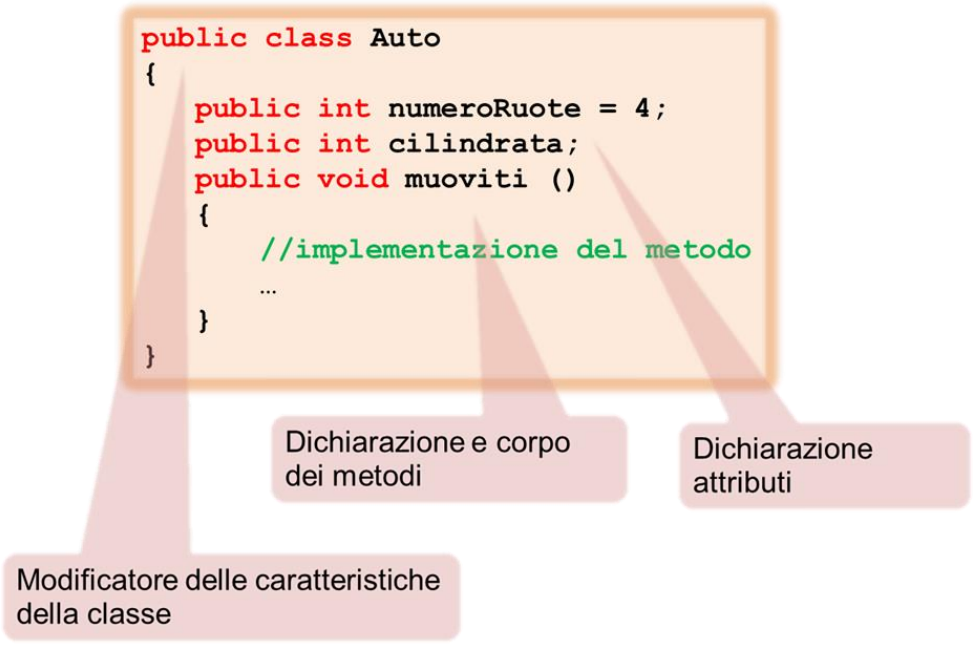


Fig. 1: Dichiarazione della classe Auto

La dichiarazione della classe Auto prevede due attributi e un metodo. Come attributo si ha un intero chiamato numeroRuote inizializzato a 4 e un altro intero chiamato cilindrata. Inoltre definisce un metodo che abbiamo chiamato muoviti() come possibile azione che può essere invocata sull'oggetto. Per il momento non ci interessa la sua implementazione.

E' una definizione piuttosto semplice ma esemplificatrice del fatto che abbiamo espresso un'astrazione del concetto «auto» in quanto ogni auto ha 4 ruote (di solito), si muove ed ha una cilindrata il cui valore non è definibile a priori ma cambia da modello a modello. Una Ferrari F430 e una Fiat 500 hanno entrambi 4 ruote, si muovono ed una cilindrata che però è diversa. La Ferrari e la Fiat sono da considerarsi oggetti ovvero istanze della classe Auto e nella realtà esisterebbero come oggetti concreti. Questa classe potrebbe costituire una parte di un'applicazione Java che, ad esempio, gestisce le auto in una concessionaria.

Possiamo salvare questo listato come Auto.java e poi compilarlo tramite l'apposito comando javac ottenendo il file Auto.class. Se mandiamo in esecuzione il bytecode otterremmo un messaggio di errore. Questo perché in questa classe è assente il metodo main() che abbiamo definito come punto di partenza dell'esecuzione di ogni applicazione Java. Eseguendo il comando java Auto, la JVM cercherebbe questo metodo per eseguirlo e non trovandolo terminerebbe istantaneamente l'esecuzione. Ciò è assolutamente in linea con la definizione di classe.

Definendo una classe Auto con codice Java, abbiamo solo definito all'interno del programma il concetto di auto, secondo la nostra interpretazione e nel contesto in cui stiamo lavorando (nel contesto del

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

problema che stiamo risolvendo per esempio un programma per la gestione di una concessionaria). Con questo codice non abbiamo in realtà definito nessuna realizzazione concreta di auto, per esempio una Ferrari F430. Come anche nella vita reale se non esistono realizzazioni concrete di un concetto (ovvero l'auto vera e propria) non può accadere nulla (non si può guidare alcunché e non ci potremmo muovere) nel codice Java la classe Auto non è eseguibile da sola.

Occorre quindi un main() e creare realizzazioni fisiche di ciò che è espresso nella definizione di classe, ovvero istanziare un oggetto o più oggetti della classe Auto.

Ciò si ottiene creando un'altra classe apposita per il programma che stiamo sviluppando (in Fig. 2 denominata Principale) che contiene il metodo main() e nel quale istanzieremo oggetto della classe Auto.

```
1.    public class Principale
2.    {
3.        public static void main (String[] args)
4.       {
5.        Auto fiat500;
6.        fiat500 = new Auto();
7.        fiat500.cilindrata = 1100;
8.        fiat500.muoviti();
9.        Auto f430 = new Auto();
10.       f430.cilindrata = 4300;
11.       f430.muoviti();

12.      }
13.    }
```

Fig. 2: Dichiarazione della classe Principale per istanziare gli oggetti e definire il flusso di esecuzione del programma con l'invocazione dei metodi sugli oggetti istanziati

Alla riga 5 è dichiarato un oggetto di tipo (appartenente alla classe) Auto che viene chiamato fiat500, ma è solamente alla riga 6 che avviene l'istanziazione (creazione) di un oggetto della classe Auto. Di fatto la parola chiava «new» istanzia la classe Auto a cui viene assegnato il nome fiat500. Dalla riga successiva è possibile utilizzare l'oggetto fiat500. Precisamente alla riga 7 si assegna all'attributo cilindrata il valore di 1100. Dopodiché invochiamo il metodo muoviti() (che per il momento non fa niente). In pratica sfruttando la definizione che ci ha fornito la classe abbiamo creato un oggetto di tipo Auto che individuato dal nome fiat500. Lo stesso dicasi per l'oggetto f430.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Classi e oggetti in un progetto in Java

In generale la classe non ha un ruolo attivo, è da considerarsi una parte strutturale dell'applicazione e quindi svolge un ruolo essenziale nell'ambito compilativo e nessun ruolo attivo nell'ambito esecutivo. Una classe si limita a definire la struttura comune di un insieme di oggetti, dichiarando gli attributi e i metodi che saranno posseduti da tutti gli oggetti che da quella classe verranno istanziati.

Nella dichiarazione di una classe, gli attributi e i metodi sono chiamati membri (denominati variabile membro e metodo membro rispettivamente).

Giacché le classi dichiarano gli attributi e metodi (senza possederli), esse non eseguono codice e non hanno un ruolo nell'ambito esecutivo. Sono infatti gli oggetti istanziati dalla classe che influenzano il flusso di lavoro del programma e quindi sono essi i protagonisti in quest'ambito secondo quando specificato nel metodo main(). In generale si usano gli attributi tramite nomeOggetto.nomeVariabile e si invocano i metodi tramte nomeOggetto.nomeMetodo().

L'operatore «.» è sinonimo di appartenenza: sono gli oggetti a possedere gli attributi dichiarati nella classe detti anche variabili di istanza (ovvero dell'oggetto) perché avranno in generale valori diversi nei due oggetti istanziati (vedi valore di cilindrata negli oggetti fiat500 e f430).

Come visto la classe Principale esegue codice contenuto all'interno dell'unico metodo main() che è assunto per default quale punto di partenza di un'applicazione Java.

Questo è vero in generale. Nel complesso un programma Java deve essere composto da una classe che contiene il metodo main(). Questa classe dovrebbe avere il nome dell'applicazione stessa (qui in generale chiamato NomeProgramma), astraendo il flusso di lavoro che deve essere eseguito. In linea teorica, quindi, la classe contenente il metodo main() non dovrebbe contenere altri metodi.

In qualche modo infatti i creatori di Java dovevano stabilire un modo per far partire l'esecuzione di un programma. La scelta è stata compiuta in base ad una questione pratica e storica: un'applicazione scritta in C o C++ ha come punto di partenza per default un programma chiamante che si deve nominare proprio main(). In Java i programmi chiamanti non esistono, ma esistono i metodi, e in particolare i metodi statici, ovvero dichiarati con static. Per il momento basti sapere che un membro dichiarato static appartiene alla classe e che tutti gli oggetti istanziati da essa convidiranno i membri (attributi o metodi) statici. Concludendo, giacché la classe NomeProgramma contiene il metodo main(), può eseguire codice.

In conclusione, come mostrato in Fig. 3 ci sarà un file contenente la classe NomeProgramma salvato come NomeProgramma.java Dopodiché abbiamo le dichiarazione delle singole classi salvati su file separati

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

(tranne qualche eccezione di cui al momento si può fare a meno sapere) anch'essi con il nome della classe e con estensione .java.

Nell'insieme questi file costituiscono il progetto del codice Java.

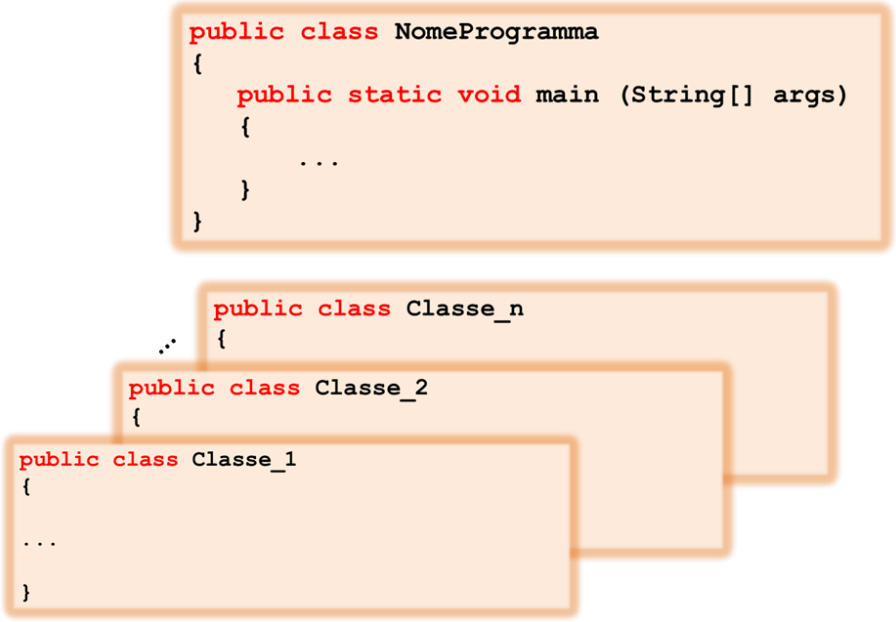


Fig. 3: Progetto in Java

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Metodi

Come detto un oggetto è una struttura dotata di propri attributi (che rappresentano il suo stato) e propri metodi (che realizzano le sue funzionalità).

Il metodo serve a definire una funzionalità di una classe e disponibile in ogni istanza (oggetto) di tale classe. Il metodo svolge delle azioni su invocazione da parte di altri oggetti o di altri metodi della classe interessati a usufruire di tale funzionalità.

Affinché un programma esegua qualche istruzione deve contenere dei metodi. Il primo fra tutti come visto è il metodo main() che per default è il punto di partenza di ogni applicazione Java. Una classe senza metodo main() come la classe Auto non può essere eseguita ma solo istanziata all'interno di un metodo di un'altra classe, incluso il main(). Il concetto di metodo è quindi anch'esso alla base della programmazione ad oggetti. Senza metodi, gli oggetti non potrebbero comunicare fra loro ovvero dare avvio al flusso di esecuzione del programma.

Anche nel caso del metodo occorre distinguere la dichiarazione di un metodo dalla chiamata di un metodo (ovvero la sua definizione e il relativo utilizzo). Vediamo innanzitutto come dichiarare un metodo.

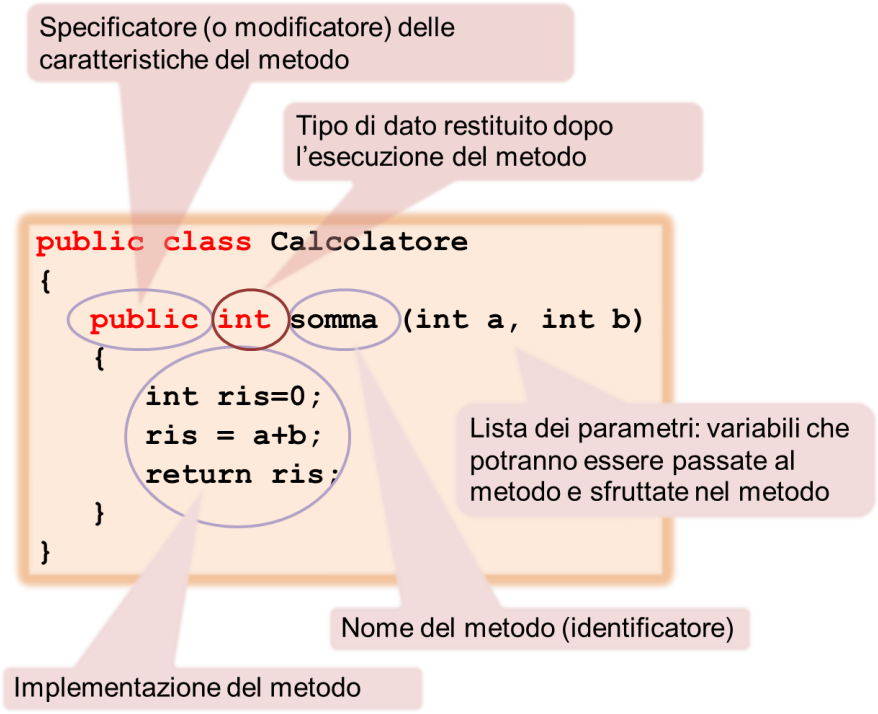


Fig. 4: Dichiarazione di un metodo

La dichiarazione di un metodo definisce la specifica dell'invocazione della funzionalità e quali azioni deve compiere il metodo quando verrà invocato per svolgere quella funzionalità. La dichiarazione di un metodo presenta la seguente sintassi mostrata tramite un esempio in Fig. 4.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La parola chiave public è una fra le tante possibili che possono essere aggiunte per specificare da chi risulta invocabile il metodo. In questo caso il metodo della classe è accessibile da oggetti esterni. Un'altra possibilità a riguardo è private che invece indica che la classe è invocabile solo da altri metodi appartenenti alla stessa classe. Altra parola chiave possibile è static (vista per il metodo main) il cui significato sarà chiarito in seguito.

Queste parole chiavi prendono il nome di modificaore o specificatore usate per modificare in qualche modo le funzionalità e le caratteristiche di un metodo.

A seguire viene specificato il tipo di dato che un metodo potrà restituire dopo essere stato chiamato. Può essere sia un tipo di dato primitivo come in questo caso che è un int, sia un tipo complesso (un oggetto) come una stringa (definita dalla classe String). E' possibile anche specificare un metodo che non restituisca nulla come nel caso del main(). In quest'ultimo caso la parola chiave da usare è void.

L'espressione che segue ogni return di un metodo deve avere un tipo compatibile con il tipo del metodo.

Dopo il tipo del dato restituito, si trova somma che è il nome del metodo detto anche identificatore del metodo. A seguire sono indicati i parametri tra parentesi tonde ovvero le dichiarazione delle variabili che potranno essere passate al metodo e di conseguenza essere sfruttate nel corpo del metodo al momento della chiamata. In questo caso a e b verranno sommati all'interno del blocco di codice del metodo e la loro somma restituita tramite il comando return come risultato finale tramite il valore di ritorno che è appunto un int. Il numero di parametri può essere zero, ma anche maggiore di uno. Se si dichiarano più parametri le loro dichiarazioni saranno separate da virgole. La coppia costituita dal nome del metodo e l'eventuale lista di parametri viene detta «firma» (in inglese «signature») del metodo.

L'insieme di istruzioni (statement) che verranno eseguite quando il metodo sarà invocato è detto corpo del metodo e costituisce la sua implementazione.

Come detto la classe Calcolatore può essere solo istanziata in un metodo di un'altra classe come può essere un main() da cui mandare in esecuzione il programma che usa la classe Calcolatore. Vediamo quindi un'altra classe eseguibile (ovvero contentente il metodo main()), che chiamiamo ancora Principale che istanzia un oggetto della classe Calcolatore e chiama il metodo somma().

In questo caso notiamo subito che l'accesso al metodo somma() è avvenuto sempre tramite l'operatore dot come nel caso dell'accesso alle variabili. Quindi tutti i membri (attributi e metodi) pubblici definiti all'interno di una classe saranno accessibili tramite un'istanza della classe stessa che sfrutta l'operatore dot. In generale la sintassi da utilizzare è la seguente: nomeOggetto.nomeMetodo(); e nomeOggetto.nomeAttributo;

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'accesso al metodo di un oggetto provoca l'esecuzione del relativo blocco di codice. Inoltre in quest'esempio abbiamo definito una variabile intera risultato che ha immagazzinato il risultato della somma. Se non avessimo fatto ciò, non avrebbe avuto senso definire un metodo che restituisse un valore dal momento che non l'avremmo utilizzato in qualche modo!

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Variabili

Nella programmazione tradizionale, una variabile è una porzione di memoria in cui è immagazzinato un certo tipo di dato. Per esempio, un intero in Java è immagazzinato in 32 bit. Vedremo in seguito i tipi di dato disponibili in Java. Serve per memorizzare i dati durante l'esecuzione di un programma o, nel caso di Java, di un metodo. In Java ogni variabile deve essere dichiarata una sola volta, la prima volta che compare nel programma e prima del suo utilizzo.

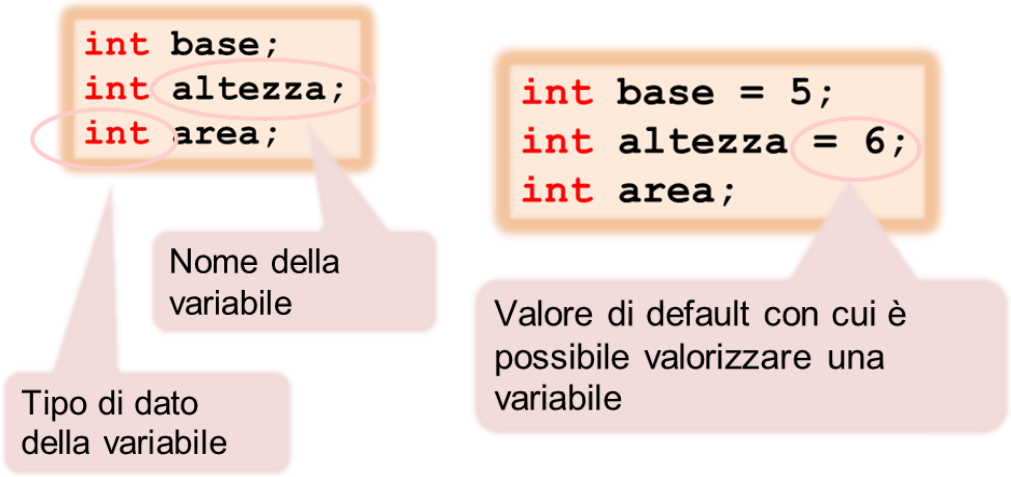


Fig. 5: Dichiarazione di variabili senza (a sinistra) e con inizializzazione (a destra)

Consideriamo il seguente esempio: int base; int altezza; int area; tre variabili di tipo intero (a 32 bit in Java) denominate rispettivamente base, altezza e area presumibilmente definite per essere usate nell'ambito di metodo che calcola l'area di un triangolo (vedi Fig. 5)). Viene specificato sia il tipo di dato che il nome della variabile detto anche identificatore. L'identificatore come il nome di una classe o di un metodo può essere costituito da lettere, numeri e underscore (esempi: num1, i, risultato, costo_tot, ...) non deve coincidere con una parola chiave del linguaggio (come class, static, int, ...) è bene che sia significativo per il programma (meglio base, altezza e area di x1, x2 e y3). La variabile può essere inizializzata al momento della dichiarazione come mostrato a destra. In questo caso dichiarazione ed assegnamento sono combinati (dichiarazione con inizializzazione) come mostrato a destra di Fig. 5. Come mostrato in Fig. 6 si può far precedere la dichiarazione di una variabile da un modificatore come per il caso della classe e del metodo che va a cambiare le caratteristiche della stessa.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

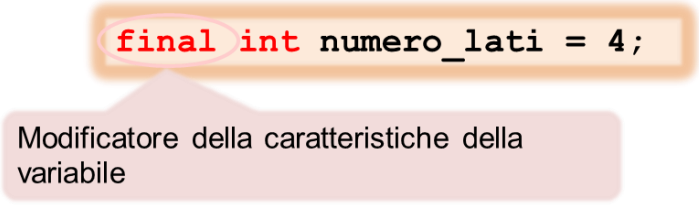


Fig. 6: Dichiarazione di una variabile con valore costante, non modificabile

Ad esempio, nella dichiarazione delle variabili che non devono mai cambiare valore si può utilizzare il modificatore final (vedi Fig. 6). Il modificatore final trasforma la variabile in una costante. Il compilatore si occuperà di controllare che il valore delle costanti non venga mai modificato (ri-assegnato) dopo essere stato inizializzato. Aggiungere il modificatore final non cambia funzionamento programma... ma serve a prevenire errori di programmazione Si chiede al compilatore di controllare che una variabile non venga ri-assegnata per sbaglio. Sapendo che una variabile non cambierà mai valore, il compilatore pu`o anche eseguire delle ottimizzazioni sull'uso di tale variabile...

Abbiamo detto che una variabile in Java deve essere dichiarata una sola volta, la prima volta che compare nel programma e prima del suo utilizzo. Infatti, anche per l'utilizzo delle variabili possiamo distinguere due fasi: dichiarazione ed assegnazione. Con l'assegnamento si intende la memorizzazione di un valore all'interno di una variabile. L'assegnazione di un valore ad una variabile è un'operazione che avviene (anche più volte) nell'ambito esecutivo (oltre che come visto contestualmente alla dichiarazione stessa). Il valore da assegnare a una variabile può essere un letterale o il risultato della valutazione di un'espressione.

Dall'esempio in Fig. 7 si può notare che le variabili possono essere usate anche all'interno di espressioni: i valori di base e altezza vengono letti e usati nell'espressione e il risultato dell'espressione viene scritto nella variabile area

```
base = 5;
altezza = 6;
area = (base+altezza)/2;
```

Fig. 7: Dichiarazione di una variabile con assegnazione tramite espressione

Il nuovo valore di una variabile può essere calcolato a partire dal precedente e aggiornato (vedi Fig. 8).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
\\ la viariabile saldo con valore
\\ precedentemente dichiarato ed assegnato
saldo = saldo + importo*1,10;
```

Fig. 8: Assegnamento di una variabile sulla base del precedente valore

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- [1] Il nuovo Java,, Claudio De Sio Cesari, Hoepli Informatica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).